```
#!/usr/bin/env python3
"""
ARCANE LANGUAGE AUDIT - PHASE 4 lexicon tools.

Mechanical checks only: phonotactic legality, a feature-based near-collision
heuristic, a minimal-rename search and a new-root generator. The distance
metric is a heuristic over articulatory features. It is NOT an acoustic or
listener test. Final acceptance still needs speaker/listener tests.
"""
import csv, json, random, sys

# ---------------------------------------------------------------- phonology
STOPS, NASALS, FRICS, LIQS, GLIDES = "pbtdkg", ["m", "n", "ŋ"], ["f", "v", "s", "
CONSONANTS = set(STOPS) | set(NASALS) | set(FRICS) | set(LIQS) | set(GLIDES)   #
VOWELS = set("aeiouə")
CODAS = set("mnŋptksʃlr")                          # z, h, w, j, b, d, g, f, v not i
NO_ONSET = {"ŋ"}                                    # ŋ is coda-only
ONSET_CLUSTERS = {"pr", "br", "tr", "dr", "kr", "gr", "kw", "gw", "tw", "fr", "fl
FRAMES = set("ptkbdgfsʃhmnŋlrwj")                 # z and v are lexical-only, not fr

PLACE = {**{c: "lab" for c in "pbmfvw"}, **{c: "alv" for c in "tdnszlr"},
         **{c: "pal" for c in "ʃj"}, **{c: "vel" for c in "kgŋ"}, "h": "glo"}
MANNER = {**{c: "stop" for c in STOPS}, **{c: "nasal" for c in NASALS},
          **{c: "fric" for c in FRICS}, **{c: "liq" for c in LIQS}, **{c: "glide"
VOICELESS = set("ptkfsʃh")
HEIGHT = {"i": 2, "u": 2, "e": 1, "o": 1, "ə": 1, "a": 0}
BACK = {"i": 0, "e": 0, "ə": 1, "a": 1, "o": 2, "u": 2}

def ph(word):
    w = word.lower().replace("sh", "ʃ").replace("ng", "ŋ")
    return list(w)

def syllabify(word):
    """Return (syllables, errors). Maximal-onset syllabification against the lega
    p, errs = ph(word), []
    nuclei = [i for i, c in enumerate(p) if c in VOWELS]
    if not nuclei:
        return [], ["no vowel"]
    for c in p:
        if c not in CONSONANTS and c not in VOWELS:
            errs.append(f"unknown phoneme {c!r}")
    sylls, prev_end = [], 0
    onsets = []
```

```
    for k, n in enumerate(nuclei):
        seg = p[prev_end:n]
        if k == 0:
            onset, coda_prev = seg, []
        else:
            if n - nuclei[k - 1] == 1:
                errs.append("adjacent vowels not allowed in lexicon")
            onset = []
            for cut in range(len(seg) + 1):                  # longest legal onse
                cand = seg[cut:]
                if len(cand) == 0 or (len(cand) == 1 and cand[0] not in NO_ONSET)
                    onset, coda_prev = cand, seg[:cut]
                    break
            if len(coda_prev) > 1:
                errs.append(f"coda cluster {''.join(coda_prev)!r}")
            sylls[-1][2] = coda_prev
        if len(onset) == 1 and onset[0] in NO_ONSET:
            errs.append("ŋ in onset")
        if len(onset) > 1 and "".join(onset) not in ONSET_CLUSTERS:
            errs.append(f"illegal onset {''.join(onset)!r}")
        sylls.append([onset, p[n], []])
        prev_end = n + 1
    tail = p[prev_end:]
    if len(tail) > 1:
        errs.append(f"coda cluster {''.join(tail)!r}")
    sylls[-1][2] = tail
    for on, nu, co in sylls:
        for c in co:
            if c not in CODAS:
                errs.append(f"illegal coda {c!r}")
    return sylls, errs

def is_foot(word):
    """A Foot is one syllable C V (C): frame consonant, parameter vowel, optional
    s, errs = syllabify(word)
    if errs or len(s) != 1: return False
    on, nu, co = s[0]
    return len(on) == 1 and on[0] in FRAMES and (not co or co[0] in CODAS)

# ------------------------------------------------------------------ distance
def csub(a, b):
    if a == b: return 0.0
    if a in VOWELS and b in VOWELS:
        return 0.75 * abs(HEIGHT[a] - HEIGHT[b]) + 0.75 * abs(BACK[a] - BACK[b])
    if (a in VOWELS) != (b in VOWELS): return 3.5
    if {a, b} == {"l", "r"}: return 1.0
```

```
    return (1.0 if PLACE[a] != PLACE[b] else 0.0) + (1.5 if MANNER[a] != MANNER[b
        + (1.0 if (a in VOICELESS) != (b in VOICELESS) else 0.0)

def dist(w1, w2, indel=1.5):
    a, b = ph(w1), ph(w2)
    D = [[0.0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(1, len(a) + 1): D[i][0] = i * indel
    for j in range(1, len(b) + 1): D[0][j] = j * indel
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            D[i][j] = min(D[i-1][j] + indel, D[i][j-1] + indel, D[i-1][j-1] + csu
    return D[-1][-1]

# ---------------------------------------------------------------- canon lexico
# (form, gloss, class, group)
CANON = []
def add(cls, group, items, status="CANON", src="v3.0-L"):
    for f, g in items:
        CANON.append(dict(form=f, gloss=g, cls=cls, group=group, status=status, s

add("DIGIT", "NUM", [(f, str(i)) for i, f in enumerate(["za","ki","to","mu","re",
add("DECIMAL", "NUM", [("shi", "decimal separator")])
add("UNIT", "NUM", [("ma","metre"),("se","second"),("ke","kilogram"),("ra","radia
add("AXIS", "AXIS", [("pa","-X"),("ba","+X"),("ta","-Y"),("da","+Y"),("ka","-Z"),
add("REF", "REF", [("mi","SELF"),("ni","FIELD"),("ri","TARGET"),("su","CURRENT"),
add("SPREL", "SPREL", [("jo","toward"),("jur","away"),("ja","along"),("je","paral
add("GEOM", "HEAD", [("pi","point"),("len","line"),("pla","plane"),("sur","surfac
                     ("par","path"),("ren","region"),("net","network")])
add("PROP", "HEAD", [("pala","position"),("vanu","velocity"),("moru","mass"),("ke
                     ("feta","force"),("enra","energy"),("teva","temperature"),("
add("INFO", "HEAD", [("nes","read"),("sima","measure"),("gama","assign"),("dave",
add("ARITH", "HEAD", [("frapa","add"),("frata","subtract"),("fraka","multiply"),(
add("VEC", "HEAD", [("dran","magnitude"),("drem","dot"),("drim","cross"),("dral",
add("LOGIC", "HEAD", [("gran","and"),("grat","or"),("grak","not")])
add("CTRL", "HEAD", [("bek","if"),("zet","while")])
add("BIND", "HEAD", [("kwam","bind"),("kwat","reference binding"),("kwak","live e
add("VERB", "HEAD", [("kren","establish"),("trak","alter"),("frem","maintain"),("
    src="Magic v3.0 §26")
add("ROLE", "ROLE", [("po","position role"),("fo","direction role"),("lo","moveme
add("COUP", "COUP", [("hə","couple: mana"),("he","couple: matter"),("ha","couple:
                     ("hu","couple: living"),("hi","couple: information"),("ho","

# ---------------------------------------------------------------- checks
def legality(entries):
    bad = []
    for e in entries:
```

```
        _, errs = syllabify(e["form"])
        if errs: bad.append((e["form"], e["gloss"], errs))
    return bad

def flagged_pairs(entries, group, thr=2.0):
    es = [e for e in entries if e["group"] == group]
    out = []
    for i in range(len(es)):
        for j in range(i + 1, len(es)):
            d = dist(es[i]["form"], es[j]["form"])
            if d < thr: out.append((es[i]["form"], es[j]["form"], round(d, 2)))
    return sorted(out, key=lambda x: x[2])

def shape(word):
    sy, errs = syllabify(word)
    return None if errs else [(len(o), len(c)) for o, n, c in sy]

def legal_variants(word):
    p = ph(word)
    seen = set()
    base = shape(word)
    for i, c in enumerate(p):
        pool = VOWELS if c in VOWELS else CONSONANTS
        for r in pool:
            if r == c: continue
            q = p[:i] + [r] + p[i+1:]
            w = "".join(q)
            if w in seen: continue
            seen.add(w)
            if not syllabify(w)[1] and shape(w) == base and "ŋ" not in w.replace(
                yield w, i, csub(c, r)

def best_rename(word, others, thr=2.0, positions=None):
    best = None
    for w, i, change in legal_variants(word):
        if positions is not None and i not in positions: continue
        md = min(dist(w, o) for o in others)
        if md >= thr:
            key = (-round(md, 2), round(change, 2), w)
            if best is None or key < best[0]: best = (key, w, md)
    return (best[1], round(best[2], 2)) if best else (None, None)

# ---------------------------------------------------------------- new roots
RESERVED_FIRST = set()   # filled from literal classes

NEW_CONCEPTS = [
```

```
    # (key, gloss, class, sibling_group)
    ("NOW", "now (instant of evaluation)", "TIME", None),
    ("BEFORE", "before", "TIME", "ba"),
    ("AFTER", "after", "TIME", "ba"),
    ("UNTIL", "until", "CTRL", None),
    ("DELAY", "delay (offset before start)", "TIME", "dd"),
    ("DURATION", "duration", "TIME", "dd"),
    ("INTERVAL", "repetition interval", "TIME", "dd"),
    ("DEADLINE", "deadline", "TIME", None),
    ("PREDICT", "predict (estimate future state)", "TIME", None),
    ("UNION", "set union", "SET", "set"),
    ("ISECT", "set intersection", "SET", "set"),
    ("DIFF", "set difference", "SET", "set"),
    ("COMPL", "set complement", "SET", "set"),
    ("FILTER", "filter set by predicate", "SET", None),
    ("ORDER", "order set by key", "SET", None),
    ("PICK", "pick min/max/nth from ordered set", "SET", None),
    ("SEARCH", "search a domain for candidates", "SET", None),
    ("ANY", "exists (any)", "SET", "q"),
    ("ALL", "for all", "SET", "q"),
    ("EACH", "for each (iteration)", "CTRL", None),
    ("REQUIRE", "hard requirement", "CONSTR", "req"),
    ("PRESERVE", "invariant: preserve", "CONSTR", "req"),
    ("WITHIN", "tolerance: within", "CONSTR", None),
    ("POWER", "exponent", "MATH", "mth"),
    ("SCI", "scientific notation (x10^n)", "MATH", "mth"),
    ("EQ", "relation: equal", "RELVAL", "rel"),
    ("LT", "relation: less than", "RELVAL", "rel"),
    ("GT", "relation: greater than", "RELVAL", "rel"),
    ("ATTACHED", "control state: attached", "STATE", "st"),
    ("TRANSFERRED", "control state: transferred", "STATE", "st"),
    ("INDEPENDENT", "control state: independent", "STATE", "st"),
    ("ONSET", "event: condition becomes true", "CTRL", None),
    ("TRANSFORM", "frame transform", "MISC", None),
    ("BEGIN", "block open", "CTRL", "blk"),
    ("END", "block close", "CTRL", "blk"),
]
ONSET_POOL = list("bdgtkmnlvwfsrp") + ["ʃ"]

def gen_roots(heads, literal_cvs, seed=4017, thr=2.0):
    rng = random.Random(seed)
    firsts = {ph(h)[0] + next(c for c in ph(h) if c in VOWELS) for h in heads}
    sib = {}
    out = {}
    pool = []
    for o1 in ONSET_POOL:
```

```
        for v1 in "aeiou":
            for o2 in ONSET_POOL:
                for v2 in "aeiou":
                    if v1 == v2: continue
                    pool.append(o1 + v1 + o2 + v2)
    rng.shuffle(pool)
    current = list(heads)
    for key, gloss, cls, sg in NEW_CONCEPTS:
        pick = None
        for w in pool:
            if w in current or w in out.values(): continue
            if syllabify(w)[1]: continue
            first = w[:2] if not w.startswith("ʃ") else w[:2]
            first = "".join(ph(w)[:2])
            if first in literal_cvs: continue
            if sg:
                ok = all(ph(w)[0] != ph(s)[0] and ph(w)[1] != ph(s)[1] for s in s
                if not ok: continue
            if min(dist(w, h) for h in current) < thr: continue
            pick = w; break
        if pick is None: raise SystemExit(f"no root found for {key}")
        out[key] = pick
        current.append(pick)
        firsts.add("".join(ph(pick)[:2]))
        if sg: sib.setdefault(sg, []).append(pick)
    return out

# ---------------------------------------------------------------------- main
if __name__ == "__main__":
    report = {}
    bad = legality(CANON)
    report["illegal_canon"] = bad
    print("Phonotactic violations in canon lexicon (with /z/ and /pl/ admitted):"

    # what the ORIGINAL inventory (no z, no pl) would have rejected
    global_z = CONSONANTS
    strict_bad = [e["form"] for e in CANON if "z" in e["form"] or e["form"] == "p
    report["rejected_by_original_inventory"] = strict_bad
    print("Forms rejected by the original 18-consonant inventory / 12-cluster lis

    heads = [e for e in CANON if e["group"] == "HEAD"]
    pairs = flagged_pairs(CANON, "HEAD")
    report["head_pairs_flagged"] = pairs
    print(f"\nHEAD near-collisions below 2.0 ({len(pairs)}):")
    for a, b, d in pairs: print(f"  {a:7s} {b:7s} {d}")
```

```
# rename loop: only sibling confusions (same class) are renamed
cls_of = {e["form"]: e["cls"] for e in CANON}
PRIO = {"frata": 5, "fraka": 6, "grak": 5, "drim": 5, "dral": 5, "kwak": 5, "
renames, accepted = {}, []
for _ in range(40):
    pr = [p for p in flagged_pairs(CANON, "HEAD") if cls_of[p[0]] == cls_of[p
    if not pr: break
    a, b, d = pr[0]
    victim = a if PRIO.get(a, 1) > PRIO.get(b, 1) else b
    partner = b if victim == a else a
    pa, pb = ph(victim), ph(partner)
    diff = {i for i in range(min(len(pa), len(pb))) if pa[i] != pb[i]}
    others = [e["form"] for e in CANON if e["group"] == "HEAD" and e["form"]
    new, md = best_rename(victim, others, positions=diff) if diff else (None,
    if new is None: new, md = best_rename(victim, others)
    if new is None:
        print("  no rename found for", victim); break
    for e in CANON:
        if e["form"] == victim:
            renames[victim] = (new, md, e["gloss"]); e["form"] = new; e["stat
report["renames"] = renames
cls_of = {e["form"]: e["cls"] for e in CANON}
accepted = [p for p in flagged_pairs(CANON, "HEAD")]
report["accepted_cross_class_pairs"] = accepted
print("\nSibling renames (old -> new, min distance to other heads):")
for old, (new, md, g) in renames.items(): print(f"  {old:7s}-> {new:7s} ({g})
print("Cross-class pairs left (type-resolved, monitored):", accepted)

# literal-class near-collisions (mitigated by prosody, not renamed)
lit = {}
for g in ("NUM", "AXIS", "SPREL", "ROLE", "COUP", "REF"):
    lit[g] = flagged_pairs(CANON, g)
    print(f"\n{g}: {len(lit[g])} pairs below 2.0 (mitigated prosodically, not
report["literal_pairs"] = {k: len(v) for k, v in lit.items()}
report["literal_pair_examples"] = {k: v[:6] for k, v in lit.items()}

# new roots
literal_cvs = {e["form"] for e in CANON if e["group"] != "HEAD" and len("".jo
heads_now = [e["form"] for e in CANON if e["group"] == "HEAD"]
roots = gen_roots(heads_now, literal_cvs)
print("\nNew roots:")
for key, gloss, cls, sg in NEW_CONCEPTS:
    print(f"  {key:12s} {roots[key]:8s} {gloss}")
report["new_roots"] = {k: roots[k] for k, *_ in NEW_CONCEPTS}

# extra literals: dimensionless unit and bit, chosen to maximise distance wit
```

```
num = [e["form"] for e in CANON if e["group"] == "NUM"]
best = []
for o in CONSONANTS - {"ŋ", "h"}:
    for v in "aeiou":
        w = o + v
        if w in num: continue
        if syllabify(w)[1]: continue
        best.append((round(min(dist(w, n) for n in num), 2), w))
best.sort(reverse=True)
report["literal_candidates"] = best[:8]
print("\nBest CV candidates for new quantity-slot literals (min distance with

# whole-lexicon accounting
from collections import Counter
report["counts"] = dict(Counter(e["cls"] for e in CANON))
report["total_canon_forms"] = len(CANON)
report["feet_valid_examples"] = {w: is_foot(w) for w in ["kan","ken","kas","k
print("\nFoot-shaped test:", report["feet_valid_examples"])

json.dump(dict(report=report, canon=CANON, roots=roots), open("/home/claude/p
          ensure_ascii=False, indent=1, default=str)
```